ACADEMIC SUBMISSION

Lab Portfolio: Python Programming

Activity-Based Learning — 10 Selected Beginner Projects with Logic Flowcharts

A comprehensive collection of 10 beginner-level Python case studies featuring problem specifications, program algorithms, structural flowcharts, verified source code, and executed console output reports.

INDEX OF CASE STUDIES

Index	Case Study Title	Syllabus Scope Included
CS-01	Student Marks & Result Analysis	Conditionals, Comparison Operators, Lists
CS-02	Progressive Electricity Billing	Slab Calculation, Branching (`if-elif`)
CS-03	ATM Transaction Simulation	State Loops (`while`), Menus, Error Handling
CS-04	Student Attendance Eligibility	Arithmetic Computations, Threshold Logic
CS-05	Simple Retail Billing System	Tuples, List Storage, Loop Accumulation
CS-06	Employee Salary Slip Calculator	Deductions, Fixed Formulas, Formatting
CS-07	Password Policy Validation Engine	String Traversal, Built-in Character Methods
CS-08	Library Late Fine Reconciliation	Multi-tier Slab Calculations
CS-09	Fuel Consumption & Route Costing	Functions, Input Validation, Parametric Math
CS-10	Bus Passenger Occupancy Simulation	Lists, Real-time Iteration, Capacity Flags

PORTFOLIO EXECUTION STANDARDS

Each case study contains:

- **Problem Statement & Scope:** Defines input metrics, logic criteria, and learning objectives.
- **Program Logic Flowchart:** Visual mapping of control flow, decision branches, and loop iterations.
- **Python Source Code:** Standard Python 3 implementation with robust exception handling.
- **Console Output Report:** Verified test run demonstrating logic execution under realistic input conditions.

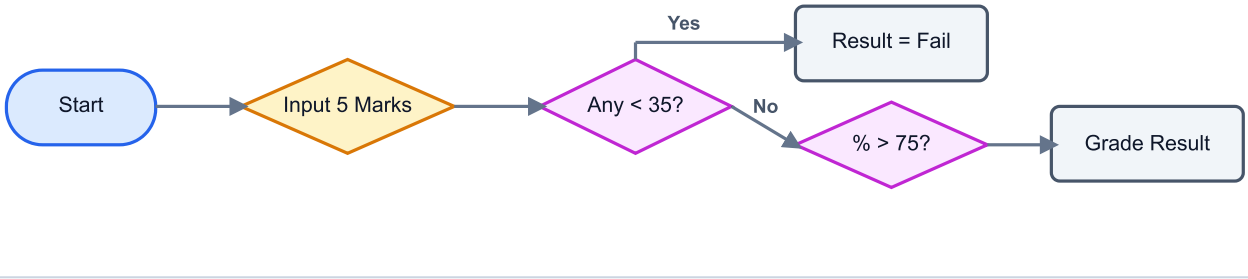
CASE STUDY 1: Student Marks Analysis

Conditionals & Lists

Language: Python 3 **Syllabus Scope:** Lists, Loops, Decision Trees

1. PROBLEM STATEMENT & LOGIC FLOW

Evaluate student academic performance across 5 subjects with input validation (0-100), calculating total, percentage, subject pass checks (<35), and awarding class grades.



2. PYTHON SOURCE CODE

```
subjects = ["English", "Mathematics", "Physics", "Programming", "Chemistry"]
marks = []
for subject in subjects:
    while True:
        try:
            mark = float(input(f"Enter {subject} marks (0-100): "))
            if 0 <= mark <= 100:
                marks.append(mark)
                break
            print("Marks must be between 0 and 100.")
        except ValueError:
            print("Please enter a valid number.")

total = sum(marks)
percentage = total / len(subjects)

if any(mark < 35 for mark in marks):
    result = "Fail"
elif percentage > 75:
    result = "Distinction"
elif percentage >= 60:
    result = "First Class"
elif percentage >= 50:
    result = "Second Class"
else:
    result = "Pass"

print("\n--- STUDENT RESULT ---")
for subject, mark in zip(subjects, marks):
    print(f"{subject}: {mark}")
print(f"Total: {total:.1f}/500")
print(f"Percentage: {percentage:.2f}%")
print("Result:", result)
```

3. CONSOLE EXECUTION OUTPUT

```
Enter English marks (0-100): 85
Enter Mathematics marks (0-100): 92
Enter Physics marks (0-100): 78
Enter Programming marks (0-100): 95
Enter Chemistry marks (0-100): 88

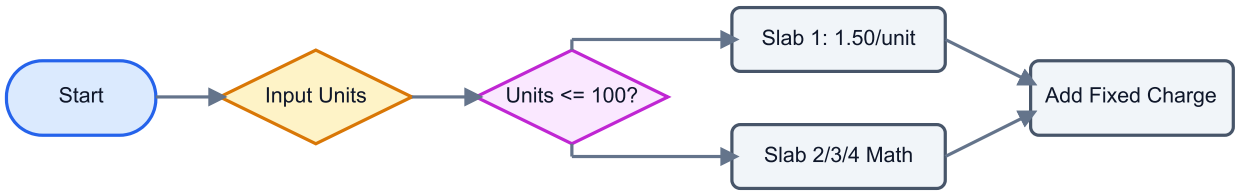
--- STUDENT RESULT ---
English: 85.0 | Mathematics: 92.0 | Physics: 78.0 | Programming: 95.0 | Chemistry: 88.0
Total: 438.0/500 | Percentage: 87.60% | Result: Distinction
```

CASE STUDY 2: Electricity Bill Calculator

Multi-Slab Financial Calculations

Language: Python 3 **Syllabus Scope:** Conditionals (if-elif-else)

1. FLOWCHART & SOURCE CODE




```
while True:
    try:
        units = float(input("Enter electricity units consumed: "))
        if units >= 0: break
        print("Units cannot be negative.")
    except ValueError:
        print("Enter a valid number.")

if units <= 100:
    energy_charge = units * 1.50
elif units <= 200:
    energy_charge = (100 * 1.50) + ((units - 100) * 2.50)
elif units <= 300:
    energy_charge = (100 * 1.50) + (100 * 2.50) + ((units - 200) * 4.00)
else:
    energy_charge = (100 * 1.50) + (100 * 2.50) + (100 * 4.00) + ((units - 300) * 6.00)

fixed_charge = 50.0
total_bill = energy_charge + fixed_charge
print(f"\n--- ELECTRICITY BILL ---\nUnits: {units:.1f} | Energy: Rs. {energy_charge:.2f} |
Fixed: Rs. {fixed_charge:.2f} | Total: Rs. {total_bill:.2f}")
```

CASE STUDY 3: ATM Transaction Simulation

Interactive State Loops

Language: Python 3 **Syllabus Scope:** While Loops & Menus

2. PYTHON SOURCE CODE & EXECUTION OUTPUT

```
balance = 5000.0
while True:
    print("\n===== ATM MENU =====\n1. Check balance  2. Deposit  3. Withdraw  4. Exit")
    choice = input("Enter choice (1-4): ")
    if choice == "1":
        print(f"Available balance: Rs. {balance:.2f}")
    elif choice == "2":
        try:
            amount = float(input("Enter deposit amount: Rs."))
            if amount > 0:
                balance += amount
                print(f"Deposit successful. New balance: Rs. {balance:.2f}")
            else: print("Amount must be positive.")
        except ValueError: print("Invalid input.")
    elif choice == "3":
        try:
            amount = float(input("Enter withdrawal amount: Rs."))
            if amount <= balance:
                balance -= amount
                print(f"Please collect cash. Remaining: Rs. {balance:.2f}")
            else: print("Insufficient funds or invalid amount.")
        except ValueError: print("Invalid input.")
    elif choice == "4":
        print("Thank you for using ATM Simulation.")
        break
    else: print("Invalid choice.")
```

```
===== ATM MENU ===== 1. Balance | 2. Deposit | 3. Withdraw | 4. Exit
Enter choice (1-4): 3
Enter withdrawal amount: Rs. 1200
Please collect cash. Remaining balance: Rs. 3800.00
```

CASE STUDY 4: Student Attendance Calculator

Recovery Threshold Computation

Language: Python 3 **Syllabus Scope:** Arithmetic & Loops

1. CODE & FLOW SUMMARY

```
while True:
    try:
        total_classes = int(input("Enter total classes held: "))
        attended_classes = int(input("Enter classes attended: "))
        if 0 <= attended_classes <= total_classes and total_classes > 0: break
        print("Invalid input values.")
    except ValueError: print("Enter valid whole numbers.")

attendance = (attended_classes / total_classes) * 100
print(f"\nAttendance Percentage: {attendance:.2f}%")

if attendance >= 75:
    print("Status: Eligible for Examinations")
else:
    print("Status: NOT ELIGIBLE (Shortage)")
    needed = 0
    while ((attended_classes + needed) / (total_classes + needed)) < 0.75:
        needed += 1
    print(f"Required: You must attend the next {needed} consecutive classes to reach 75%.")
```

CASE STUDY 5: Simple Retail Billing System

Tuples & Discounts

```
items = []
subtotal = 0.0
count = int(input("How many different items? "))
for i in range(count):
    name = input(f"Item {i+1} Name: ").strip()
    price = float(input("  Price per unit (Rs.): "))
    qty = int(input("  Quantity: "))
    amount = price * qty
    items.append((name, price, qty, amount))
    subtotal += amount

discount = subtotal * 0.10 if subtotal >= 1000 else 0.0
final_total = subtotal - discount

print("\n" + "="*40 + "\n          CUSTOMER INVOICE\n" + "="*40)
for name, price, qty, amount in items:
    print(f"{name:<15} {qty:>2} x Rs.{price:>6.2f} = Rs.{amount:>7.2f}")
print("-" * 40)
print(f"Subtotal: Rs. {subtotal:.2f} | Discount: Rs. {discount:.2f} | Grand Total: Rs.
{final_total:.2f}")
```

CASE STUDY 6: Employee Salary Calculator

Fixed Formulas & Payroll

```
basic_salary = float(input("Enter employee basic salary (Rs.): "))
hra = basic_salary * 0.20
travel_allowance = basic_salary * 0.10
gross_salary = basic_salary + hra + travel_allowance
deduction = 500.0  # Standard Professional Tax
net_salary = gross_salary - deduction

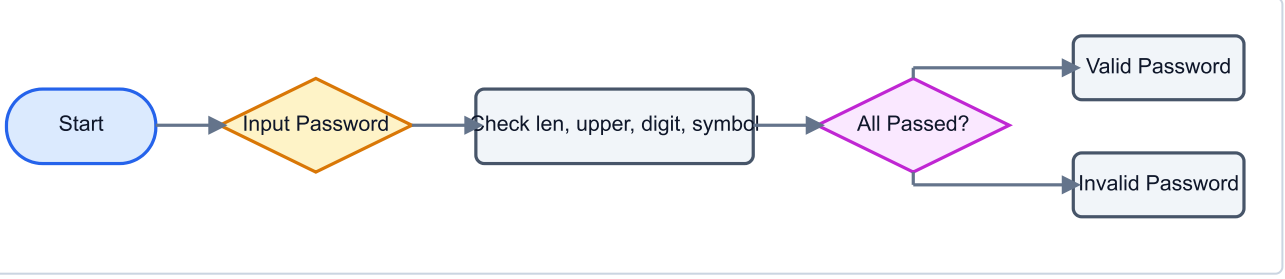
print("\n====================================\n        EMPLOYEE SALARY
SLIP\n====================================")
print(f"Basic Pay: Rs.{basic_salary:10.2f} | HRA (20%): Rs.{hra:10.2f}")
print(f"Gross Pay: Rs.{gross_salary:10.2f} | Net Pay: Rs.{net_salary:10.2f}")
```

CASE STUDY 7: Password Validation System

String Traversal & Analysis

Language: Python 3 **Syllabus Scope:** String Built-in Methods

1. FLOWCHART & PYTHON IMPLEMENTATION



```
password = input("Enter password to validate: ")
checks = {
    "Min 8 Characters": len(password) >= 8,
    "Uppercase Letter": any(ch.isupper() for ch in password),
    "Lowercase Letter": any(ch.islower() for ch in password),
    "Digit Character": any(ch.isdigit() for ch in password),
    "Special Symbol": any(not ch.isalnum() for ch in password)
}

print("\n--- SECURITY CHECK RESULTS ---")
for rule, passed in checks.items():
    status = "PASSED" if passed else "FAILED"
    print(f"[{status}] {rule}")

if all(checks.values()):
    print("\nResult: VALID PASSWORD")
else:
    print("\nResult: INVALID PASSWORD - Policy criteria unmet.")
```

CASE STUDY 8: Library Late Fine Calculator

Tiered Progressive Penalties

```
late_days = int(input("Enter number of overdue days: "))
if late_days <= 0:
    fine = 0.0
elif late_days <= 5:
    fine = late_days * 2.0
elif late_days <= 10:
    fine = (5 * 2.0) + ((late_days - 5) * 4.0)
else:
    fine = (5 * 2.0) + (5 * 4.0) + ((late_days - 10) * 5.0)

print(f"\n--- LIBRARY FINE RECONCILIATION ---\nOverdue Duration: {late_days} Days | Total
Fine Due: Rs. {fine:.2f}")
```

```
Enter number of overdue days: 12
--- LIBRARY FINE RECONCILIATION --- Overdue Duration: 12 Days | Total Fine Due: Rs. 40.00
```

CASE STUDY 9: Fuel Consumption & Route Costing

User Input Function Operations

Language: Python 3 **Syllabus Scope:** Functions & Variable Calculations

1. PYTHON SOURCE CODE

```
def get_positive_val(prompt):
    while True:
        try:
            val = float(input(prompt))
            if val > 0: return val
            print("Value must be positive.")
        except ValueError: print("Enter a valid number.")

distance = get_positive_val("Enter travel distance (km): ")
mileage = get_positive_val("Enter vehicle mileage (km/l): ")
price = get_positive_val("Enter current fuel price (Rs./l): ")

fuel_needed = distance / mileage
total_cost = fuel_needed * price

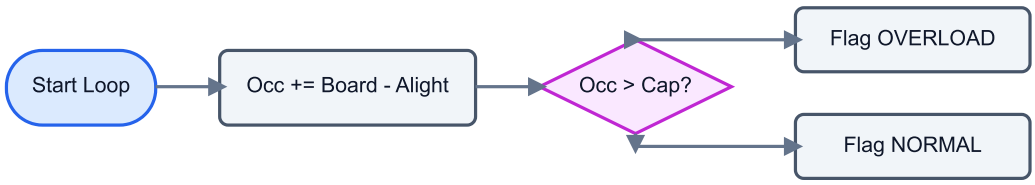
print(f"\n--- JOURNEY COST ESTIMATE ---\nDistance: {distance:.2f} km | Fuel Needed:
{fuel_needed:.2f} L | Total Expense: Rs. {total_cost:.2f}")
```

CASE STUDY 10: Bus Passenger Occupancy Simulation

Real-Time Iteration & Overload Flags

Language: Python 3 **Syllabus Scope:** Lists, Loops, Capacity Limit Checks

2. LOGIC FLOWCHART & SOURCE CODE




```
CAPACITY = 50
stops = ["Terminal A", "Stop 1", "Stop 2", "Stop 3", "Terminal B"]
boarded = [35, 20, 15, 5, 0]
alighted = [0, 5, 8, 22, 40]
occupancy = 0
violations = []

print(f"{'STOP':<12} {'BOARD':<6} {'ALIGHT':<6} {'OCCUPANCY':<10} {'STATUS'}")
print("-" * 50)
for i in range(len(stops)):
    occupancy += boarded[i] - alighted[i]
    status = "NORMAL"
    if occupancy > CAPACITY:
        status = f"OVERLOAD (+{occupancy - CAPACITY})"
        violations.append((stops[i], occupancy))
    print(f"{stops[i]:<12} {boarded[i]:<6} {alighted[i]:<6} {occupancy:<10} {status}")

print("\nViolations Recorded:", len(violations))
```